BlockBuildDocumentation

Author: Siyana Bandrova, Scientific Supervisor: Radostin Cholakov

January 2026

1 Layers

In the following section, the characteristics of each layer in BlockBuild are presented.

1.1 Linear

The Linear layer (also known as Dense) multiplies the input vector by a matrix W and adds a bias b.

For $x = [x_1, x_2, x_3]$, $x' = W \cdot x + b$

In: number of input features

Out: number of output features

1.2 ReLU

ReLU (Rectified Linear Unit) is a nonlinear function that converts all negative numbers to 0.

$x = \max(x, 0)$

It is used to prevent very negative values and allows disabling some parameters. This enables the neural network to learn more complex data. It is not trainable.

1.3 Dropout

Dropout disables some parameters for one training step according to a given probability. This non-parametric layer helps reduce overfitting, allowing the model to be trained using all the data rather than only subsets of it.

It is recommended to use the following dropout rates (in percent):
10 – Not enough impact
20 – Recommended
30–40 – Used for larger models
50 – Used for more complex tasks and generally not recommended

Probability: probability of disabling a parameter

1.4 LayerNorm

LayerNorm (Layer Normalization) stabilizes the input data by normalizing it using the following formula:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

where μ is the mean of the elements used for normalization,

$$\mu = \frac{1}{N} \sum_{i=1}^{N} x_i$$

the variance is

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu)^2$$

and ϵ is a small number added to avoid division by zero.

Normalized Shape: size of the data in the last dimension

1.5 BatchNorm

BatchNorm (Batch Normalization) normalizes the input data across the batch dimension, which helps stabilize and accelerate training.

For an input x, the normalized value is computed as:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

where μ_B is the mean of the batch:

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

and the variance is:

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2$$

After normalization, BatchNorm applies a linear transformation:

$$y = \gamma \hat{x} + \beta$$

where γ (scale) and β (shift) are learnable parameters.

BatchNorm reduces internal covariate shift and allows the use of higher learning rates. It is a trainable layer.

Num Features: number of channels/features to normalize

1.6 Embedding

The Embedding layer converts data (e.g., indices) into a vector of numbers that the neural network can use. This trainable layer is represented as a matrix, where each row corresponds to the vector of a given token.

Embedding Dim: number of values in each embedding vector

Num Embeddings: number of distinct indices

1.7 Convolution

The Conv layer slides over the vector, taking groups of a fixed size and multiplying them by a filter (kernel), which is a vector of numbers. Its parameters are trained during backpropagation.

Example: Let the initial list be $[2, 4, 5, 3, 1]$ with kernel $[1, 0, -1]$
Step 1: $2 \cdot 1 + 4 \cdot 0 + 5 \cdot (-1) = -3$
Step 2: $4 \cdot 1 + 5 \cdot 0 + 3 \cdot (-1) = 1$
Step 3: $5 \cdot 1 + 3 \cdot 0 + 1 \cdot (-1) = 4$
Output vector: $[-3, 1, 4]$

Kernel Size: length of each filter

In Channels: number of input channels

Out Channels: number of output filters

Stride: shift after each step

Padding: number of zeros added to the beginning or end of the vector

1.8 Convolution Transpose

ConvTranspose is the inverse operation of Conv. While Conv reduces the size of the output vector, Transpose increases its dimensionality. ConvTranspose is a trainable layer that multiplies each element of the input vector by a filter and sums the overlapping results (where two or more kernels cover the same output position).

Example: Input vector $[2, 4, 5]$; filter $[1, 0, -1]$, kernel size $= 3$, stride $= 1$
Step 1: $2 \cdot [1, 0, -1] = [2, 0, -2]$ – current output vector
Step 2: $4 \cdot [1, 0, -1] = [4, 0, -4]$; added with overlap $\rightarrow [2, 4, -2, -4]$
Step 3: $5 \cdot [1, 0, -1] = [5, 0, -5]$; added with shift 2 $\rightarrow [2, 4, 3, -4, -5]$

Kernel Size: length of the filter

Stride: shift after each step

Padding: number of zeros added to the beginning or end of the vector

1.9 AveragePooling

AvgPool is a layer that reduces data length by computing the average for each segment of kernel size elements and adding it to the output vector.

Example: Input vector $[1, 2, 3, 4, 5, 6]$, kernel size $= 2$
Step 1: $[1, 2] \Rightarrow (1 + 2)/2 = 1.5$
Step 2: $[3, 4] \Rightarrow (3 + 4)/2 = 3.5$
Step 3: $[5, 6] \Rightarrow (5 + 6)/2 = 5.5$
Output vector: $[1.5, 3.5, 5.5]$

Kernel Size: length of the filter

Stride: shift after each step

Padding: number of zeros added to the input vector

1.10 MaxPooling

MaxPool, like AvgPool, reduces the dimensionality of the input vector by splitting it into groups of kernel size elements and selecting the maximum element from each group.

Example: Input vector $[1, 5, 2, 7, 3, 4]$, kernel size $= 2$, padding $= 2$
After padding: $[0, 0, 1, 5, 2, 7, 3, 4, 0, 0]$
1) $[0, 0] \rightarrow$ max: 0 2) $[1, 5] \rightarrow$ max: 5 3) $[2, 7] \rightarrow$ max: 7 4) $[3, 4] \rightarrow$ max: 4 5) $[0, 0] \rightarrow$ max: 0

Output vector: $[0, 5, 7, 4, 0]$

Kernel Size: length of the filter

Stride: shift after each step

Padding: number of zeros added to the input vector

1.11 AdaptiveAveragePooling

AdaptiveAvgPool is a type of AveragePooling layer that automatically selects the kernel and stride to achieve the desired output size.

Output Size: length of the output vector

Note: The formula for calculating the output size for Conv, ConvTranspose, MaxPool, AvgPool, and AdaptiveAvgPool is:

$$OutputSize = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1$$

$$N = inputsize, \quad K = kernelsize, \quad S = stride, \quad P = padding$$

1.12 Concat

Concat (Concatenation) combines two or more tensors. It does not change the values of the data but increases the size of the selected dimension. This layer is not trainable.

Example: Vector A: $[1, 2, 3]$ Vector B: $[4, 5, 6]$ Concatenated output: $[1, 2, 3, 4, 5, 6]$

Ports: number of tensors which should be combined

1.13 Softmax

Softmax converts a vector of real numbers into a probability distribution. Each value becomes a number between 0 and 1, and the sum of all outputs equals 1.

$$Softmax(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{N} e^{x_j}}$$

This layer is typically used in classification tasks as the final layer. It is not trainable.

Dim: dimension along which softmax is applied

Example 1: dim = 0: Sample tensor: [[1, 2, 3], [4, 5, 6]] => [1+4, 2+5, 3+6] = [5, 7, 9]

Example 2: dim = 1: Sample tensor: [[1, 2, 3], [4, 5, 6]] => [1+2+3, 4+5+6] = [6, 15]

1.14 MatMul

MatMul (Matrix Multiplication) multiplies two matrices according to the rules of linear algebra.

If $A \in R^{m \times n}$ and $B \in R^{n \times p}$, then:

$$C = A \cdot B \in R^{m \times p}$$

This layer is commonly used in attention mechanisms and transformer architectures. It is not trainable by itself but operates on trainable tensors.

Requires: compatible inner dimensions

1.15 Scale

Scale multiplies the input tensor by a scalar value.

$$y = x \cdot \alpha$$

where α is a constant or learnable parameter.

This operation is commonly used in attention mechanisms (e.g., scaling by $1/\sqrt{d_k}$).

Type	Description	PyTorch Equivalent
linear	Linear transformation: $y = Wx + b$	nn.Linear
relu	Activation function: $y = \max(x, 0)$	torch.relu
dropout	Randomly zeros neurons during training.	nn.Dropout
layernorm	Feature-wise normalization: $y_i = \frac{x_i - \mu}{\sigma}$	nn.LayerNorm
batchnorm	Batch-wise normalization: $y_i = \frac{x_i - \mu_B}{\sigma_B}$	nn.BatchNorm1d/2d/3d
conv	Convolution operation: $y_{\mathbf{i}} = \sum_{\mathbf{u}} x_{\mathbf{i}\cdot\mathbf{s}+\mathbf{u}-\mathbf{p}}\, w_{\mathbf{u}}$	nn.Conv1d/2d/3d
convtranspose	Transposed convolution: $y_{\mathbf{i}} = \sum_{\mathbf{u}} x_{\mathbf{i}+\mathbf{p}-\mathbf{u}}\, w_{\mathbf{u}}$ $onlywhen(\mathbf{i}+\mathbf{p}-\mathbf{u}) \equiv 0 \pmod{\mathbf{s}}$	nn.ConvTranspose1d/2d/3d
maxpool	Max pooling: $y_i = \max_{u\in[0,k-1]} x_{i\cdot s+u}$	nn.MaxPool1d/2d/3d
avgpool	Average pooling: $y_i = \frac{1}{\lvert\mathcal{K}\rvert} \sum_{u\in\mathcal{K}} x_{\,i\cdot s+u-p}$	nn.AvgPool1d/2d/3d
adaptiveavgpool	Adaptive average pooling: $y_j = \dfrac{1}{\lvert I_j\rvert} \displaystyle\sum_{u\in I_j} x_u$	nn.AdaptiveAvgPool1d/2d/3d
embedding	Vector representation of indices: $y_t = W_{x_t}$	nn.Embedding
concat	Tensor concatenation: $y_i = [x_i^{(1)} \| x_i^{(2)} \| \ldots \| x_i^{(k)}]$	torch.cat
softmax	Probability normalization: $y_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$	torch.softmax
matmul	Matrix multiplication: $y_{ij} = \sum_k x_{ik} w_{kj}$	torch.matmul
scale	Scaling: $y_i = \alpha x_i$	x * α
mask	Masking values: $Y = X \odot M$	x * mask

Таблица 1: Blocks and their corresponding PyTorch equivalents

2 Summary - table

Note: General notation: i — output index, u — kernel index, s — stride, p — padding, w_u — kernel weights, $\mathcal{K}$ — set of positions in the pooling window, x — input, y — output. AdaptiveAvgPool: I — input interval averaged for a given output position. Embedding: N — number of tokens, D — embedding dimension.

3 Layer Interactions (Connections)

The model represents an acyclic graph where the edges reflect the sequence of functions. Connections between nodes with different dimensions or mismatches between the source output and target input are invalid. In such cases, the system generates an error and removes the invalid edge.

4 Example Model

- Linear (In: 2, Out: 16) -> ReLU -> Linear (In: 16, Out: 1) ; Sample dataset: dataset.01.txt;
- Conv1d(InChannels: 1, OutChannels: 8, Kernel: 3, Stride: 1, Padding: 1) -> ReLU -> Conv1d(InChannels: 8, OutChannels: 16, Kernel: 3, Stride:

1, Padding: 1) -> Linear (In: 128, Out: 32) -> ReLU -> Linear (In: 32, Out: 1); Sample dataset: dataset.02.txt

- Linear (In: 4, Out: 64) -> ReLU -> LayerNorm (Normalized Shape: 64) -> Linear (In: 64, Out: 128) -> ReLU -> Linear (In: 128, Out: 64) -> ReLU -> LayerNorm (Normalized Shape: 64) -> Linear (In: 64, Out: 1) Sample dataset: dataset.03.txt

All datasets used in the example models are publicly available on GitHub. Link: https://github.com/siyanabandrova-dotcom/BlockBuild